

Java 基础

什么是 Java?

Java 是一种通用、基于类、面向对象的编程语言，最初于 1995 年发布。Java 语言规范、Java SE 平台和具体的 JDK 发行版是不同概念；Oracle JDK、OpenJDK 及其他厂商发行版都可以提供 Java 开发工具和运行环境。

它可用于：

- 移动应用程序（尤其是 Android 应用）
- 桌面应用程序
- Web 应用程序
- Web 服务器和应用服务器
- 游戏
- 数据库连接
- 以及更多、更多用途！

为什么使用 Java?

- Java 可在不同平台上运行（Windows、Mac、Linux、Raspberry Pi 等）
- 它具有成熟的标准库、构建工具、测试工具和第三方生态系统
- 静态类型检查可以在编译期发现许多错误
- 自动内存管理减少了手动管理内存的负担
- Java 的主要参考实现 OpenJDK 是开源的；具体 JDK 发行版的许可条款可能不同
- JVM 提供异常处理、运行时检查、即时编译和跨平台运行能力；实际性能与可靠性仍取决于程序设计和运行环境
- 它拥有庞大的开发者社区和丰富的学习资料
- Java 是一种基于类的面向对象语言，它可以为程序提供清晰结构并支持代码复用，从而降低开发和维护成本
- 由于 Java 与 C++ 和 C# 相近，程序员可以轻松地转向 Java，反之亦然

为什么在 FRC 编程中使用 Java?

Java 是 FRC 官方支持的文本编程语言之一。[WPILib 官方文档](#)同时列出 Java、C++ 和 Python，并建议编程经验较少的新队员优先考虑 Java。Java 在易学性、开发便利性和机器人控制所需性能之间取得了较好的平衡。

FRC 中经常选择 Java 的主要原因包括：

- **与高中课程衔接：**Java 是 AP Computer Science A 和许多编程入门课程使用的语言。学生可以将课堂中学到的知识直接应用到机器人项目中。
- **自动内存管理：**Java 通过垃圾回收管理对象内存，避免了 C++ 中常见的悬空指针和手动释放错误。但垃圾回收并不能消除所有内存泄漏，也不能保证程序不会在比赛中崩溃；机器人代码仍需控制对象分配、处理异常并进行充分测试。
- **开发便利性：**Java 通常能提供较快的编译、部署和调试反馈，并由编译器和运行时执行大量类型及边界检查。实际构建与部署时间仍取决于项目规模、Gradle 缓存和计算机性能。
- **标准 FRC 生态系统支持：**`**WPILib**` 是 WPILib 的 Java 版本，提供机器人硬件接口、Command-Based 框

架、数学与控制、仿真和调试工具。

- **社区资源丰富：**Java 在 FRC 社区中使用广泛，因此在 WPILib 文档、官方示例、*Chief Delphi* 和其他队伍中通常容易找到相关资料。这里不使用未经官方统计支持的具体队伍占比。

重要（版本基准）：本文的 FRC 版本说明以 WPILib 2026 稳定版为基准，其机器人项目使用 Java 17。WPILib 2027 开发版已迁移到 Java 25，因此不要跨赛季手工沿用 JDK、GradleRIO 或 vendor library 配置；应使用目标赛季的 WPILib Installer 和项目模板。

安装 Java

某些 PC 可能已经安装了 Java。编译 Java 源代码需要 JDK，而不仅是只能运行程序的 JRE。

要检查 Windows PC 上是否已安装 Java，请在开始栏中搜索 Java，或在命令提示符（cmd.exe）中输入以下命令：

```
C:\Users\Your Name>java -version
```

如果已安装 Java，你将看到类似以下内容（版本号、发行版名称和输出格式会有所不同）：

```
openjdk version "17.0.12" 2024-07-16 LTS
OpenJDK Runtime Environment (build 17.0.12+7-LTS)
OpenJDK 64-Bit Server VM (build 17.0.12+7-LTS, mixed mode, sharing)
```

如果只是学习通用 Java，可以从 [Oracle Java Downloads](#) 或其他可信的 OpenJDK 发行版获取 JDK。对于 FRC 项目，应优先使用 [WPILib Installer](#) 随当赛季安装的 JDK，以避免本机 JDK 版本与 GradleRIO、WPILib 工具或 roboRIO 运行环境不兼容。

Java 快速入门

在本文采用的传统 Java 源文件结构中，程序入口位于类中。若一个顶层类声明为 `public`，其类名必须与源文件名一致；非 `public` 顶层类不受此文件名规则限制。FRC 的 WPILib 项目由模板和 GradleRIO 管理入口类，不需要手工套用本节的命令行目录结构。

让我们创建第一个 Java 文件，将其命名为 `Main.java`；可以使用任何文本编辑器（例如记事本）完成此操作。

该文件应包含一条“Hello World”消息，代码如下：

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello world");
    }
}
```

如果你不理解上面的代码，不必担心——我们会在后续章节中详细讨论。目前，请重点关注**如何**运行上述代码。

在记事本中将代码保存为“Main.java”。打开命令提示符（cmd.exe），转到保存文件的目录，然后输入“`javac Main.java`”：

```
C:\Users\Your Name>javac Main.java
```

这将编译你的代码。如果代码中没有错误，命令提示符将进入下一行。现在，输入“`java Main`”运行该文件：

```
C:\Users\Your Name>java Main
```

输出应为：

```
Hello world
```

示例说明

在本示例采用的传统结构中，可执行语句位于类的方法内。类名按 Java 命名惯例应以大写字母开头。在本示例中，我们将类命名为 **Main**。

说明：Java 区分大小写。`MyClass` 和 `myclass` 会被视为两个完全不同的名称。

包含 `public class Main` 的 Java 文件名**必须是** `Main.java`。这是 Java 对 `public` 顶层类型的命名要求；如果名称不一致，编译器会报错。

保存文件时，请使用类名作为文件名，并在文件名末尾添加 `.java`。

main 方法

在本文和传统 FRC Java 项目采用的写法中，可直接启动的 Java 应用程序使用以下 `main()` 方法作为入口：

```
public static void main(String[] args)
```

放在 `main()` 方法中的任何代码都会被执行。

目前，你不需要理解关键字 `public`、`static` 和 `void`。本教程稍后会介绍它们。只需记住：对于本文的传统写法，JVM 从 `main()` 入口开始执行程序。较新的 Java 版本还支持更简化的入口形式，但 FRC 代码应遵循所用 WPILib 版本生成的项目模板。

System.out.println()

在 `main()` 方法中，可以使用 `println()` 方法向屏幕打印一行文本：

```
public static void main(String[] args) {  
    System.out.println("Hello world");  
}
```

说明：花括号 `{}` 标记一个代码块的开始和结束。

`System.out.println()` 看起来很长，但你可以把它看作一条命令，意思是：“**将这段文本发送到屏幕。**”

以下是各部分的含义（稍后会学习详细内容）：

- `System` 是 Java 的内置类。
- `out` 是 `System` 的成员，是“output”的缩写。
- `println()` 是一个方法，是“print line”的缩写。

多数 Java 语句必须以分号（`;`）结尾，例如变量声明、赋值、方法调用和 `return` 语句。类、方法和 `if` 等代码块本身通常不在右花括号后添加分号。遗漏所需的分号会导致编译错误。

字符串字面量必须包含在双引号 `""` 内；单个 `char` 字面量使用单引号，例如 `'A'`。遗漏引号或混用引号会导致编译错误。

print() 方法

还有一个与 `println()` 类似的 `print()` 方法。

唯一的区别是，它不会在输出末尾插入换行符：

```
System.out.print("Hello world! ");
System.out.print("I will print on the same line.");
```

Java 注释

单行注释

单行注释以两个正斜杠 (`//`) 开头。

`//` 与行末之间的任何文本都会被 Java 忽略（不会执行）。

此示例在一行代码前使用了单行注释：

```
// This is a comment
System.out.println("Hello world");
```

Java 多行注释

多行注释以 `/*` 开头，以 `*/` 结尾。

`/*` 与 `*/` 之间的任何文本都会被 Java 忽略。

此示例使用多行注释（注释块）解释代码：

```
/* The code below will print the words Hello world
to the screen, and it is amazing */
System.out.println("Hello world");
```

Java 变量

变量是存储数据值的容器。

在 Java 中，变量有不同的**类型**，例如：

- `String` —— 存储文本，例如“Hello”。String 值由双引号包围
- `int` —— 存储不含小数的整数，例如 123 或 -123
- `float` —— 存储带小数的浮点数，例如 19.99 或 -19.99
- `char` —— 存储单个字符，例如 'a' 或 'B'。char 值由单引号包围
- `boolean` —— 存储两种状态的值：true 或 false

声明（创建）变量

要在 Java 中创建变量，需要：

- 选择一个**类型**（例如 `int` 或 `String`）
- 为变量指定一个**名称**（例如 `x`、`age` 或 `name`）
- 可选择使用 `=` 为其**赋值**

例如，如果要存储一些文本，可以使用 `String`：

```
String name = "John";
```

要创建一个用于存储数字的变量，可以使用 `int`：

```
int myNum = 15;
```

Final 变量

如果不希望变量在完成初始化后再次赋值，请使用 `final` 关键字：

```
final int myNum = 15;  
myNum = 20; // will generate an error: cannot assign a value to a final variable
```

当变量值绝不应改变时，应将其声明为 `final`。例如，一小时的分钟数或你的出生年份：

```
final int MINUTES_PER_HOUR = 60;  
final int BIRTHYEAR = 1980;
```

`final` 限制的是变量的再次赋值。若 `final` 变量保存对象引用，不能让该变量重新指向另一个对象，但原对象的可变状态仍可能改变。只有满足编译期常量表达式等条件的 `static final` 或 `final` 基本类型/`String` 变量，才是 Java 语言意义上的常量变量。

混合文本与数字

在 Java 中，`+` 符号有两种含义：

- 对于文本（字符串），它会将文本连接在一起（称为**拼接**）。
- 对于数字，它会将数值相加。

对于数值，`+` 字符作为数学运算符使用（请注意，此处使用的是 `int`（整数）变量）：

```
int x = 5;  
int y = 6;  
  
System.out.println("The sum is " + x + y); // Prints: The sum is 56  
System.out.println("The sum is " + (x + y)); // Prints: The sum is 11
```

说明：

在第一行中，Java 将 `"The sum is "` 与 `x` 组合，创建字符串 `"The sum is 5"`。随后将 `y` 添加到该字符串，因此它变为 `"The sum is 56"`。

在第二行中，括号确保先计算 `x + y`（结果为 `11`），因此输出为 `"The sum is 11"`。

Java 标识符

Java 中的变量、方法、类等程序元素都使用名称进行标识，这些名称称为**标识符**。同一作用域内不能声明相互冲突的变量名。

标识符可以是较短的名称（如 x 和 y），也可以是更具描述性的名称（age、sum、totalVolume）。

说明：建议使用具有描述性的名称，以创建易于理解和维护的代码：

```
// Good
int minutesPerHour = 60;

// OK, but not so easy to understand what m actually is
int m = 60;
```

变量命名的一般规则如下：

- 标识符可以包含 Java 允许的 Unicode 字母和数字，以及下划线和美元符号；日常代码通常使用英文字母、数字和下划线
- 名称必须以 Java 字母、\$ 或 _ 开头，不能以数字开头；从 Java 9 起，单独的 _ 不能作为标识符
- 按 Java 命名惯例，变量名和方法名通常以小写字母开头并采用 lowerCamelCase；名称不能包含空白字符
- 名称区分大小写（“myVar”和“myvar”是不同的变量）
- 关键字和受限标识符在其受限上下文中不能用作名称，例如 `int`、`boolean` 和局部变量类型位置中的 `var`

以下是一些会导致错误的**无效**标识符示例：

```
// Invalid identifiers:
int 2ndNumber = 5; // Cannot start with a digit
int my var = 10;   // Cannot contain spaces
int int = 20;      // Cannot use reserved keywords
```

Java 数据类型

如上一章所述，Java 中的变量必须具有指定的数据类型。

数据类型分为两组：

- 基本数据类型——包括 `byte`、`short`、`int`、`long`、`float`、`double`、`boolean` 和 `char`
- 非基本数据类型——例如 `String`、[数组](#)和[类](#)（后续章节将进一步介绍这些内容）

基本数据类型

基本数据类型规定变量的类型以及它可以保存的值的种类。

Java 中有八种基本数据类型：

数据类型	说明
<code>byte</code>	存储 -128 到 127 的整数
<code>short</code>	存储 -32,768 到 32,767 的整数

数据类型	说明
<code>int</code>	存储 -2,147,483,648 到 2,147,483,647 的整数
<code>long</code>	存储 -9,223,372,036,854,775,808 到 9,223,372,036,854,775,807 的整数
<code>float</code>	32 位 IEEE 754 单精度浮点数，通常约有 6 到 7 位十进制有效数字
<code>double</code>	64 位 IEEE 754 双精度浮点数，通常约有 15 到 16 位十进制有效数字
<code>boolean</code>	存储 true 或 false 值
<code>char</code>	存储一个 16 位 UTF-16 代码单元；并非所有 Unicode 字符都能由单个 <code>char</code> 表示

变量一旦以某种类型声明，在程序后续部分中就不能更改为另一种类型。如果确实需要在类型之间转换，必须使用[类型转换](#)或转换方法（例如，将 `int` 转换为 `double`）。

基本数值类型分为两组：

整数类型存储不含小数的正整数或负整数（例如 123 或 -456）。有效类型为 `byte`、`short`、`int` 和 `long`。应使用哪种类型取决于数值大小。

浮点类型表示带有小数部分、包含一位或多位小数的数值。有两种类型：`float` 和 `double`。

Java 中带小数点的浮点字面量默认为 `double`。赋给 `float` 时通常需要 `f` 或 `F` 后缀，例如 `float value = 19.99f;`。浮点数采用二进制表示，许多十进制小数无法精确表示，因此不应直接使用 `==` 比较一般计算结果；对于货币等要求精确十进制运算的场景，应考虑 `BigDecimal`。

尽管 Java 中有许多数值类型，但最常用于数字的是 `int`（用于整数）和 `double`（用于浮点数）。

Java boolean 数据类型

在编程中，经常需要一种只能具有以下两种值之一的数据类型：

- YES / NO
- ON / OFF
- TRUE / FALSE

为此，Java 提供了 `boolean` 数据类型，它只能取值 `true` 或 `false`：

```
boolean isJavaFun = true;
boolean isFishTasty = false;
System.out.println(isJavaFun);    // Outputs true
System.out.println(isFishTasty);  // Outputs false
```

非基本数据类型

Java 规范通常使用[引用类型（reference type）](#)这一名称。引用类型包括类类型、接口类型和数组类型；`String` 只是类类型中的一个常用示例。

基本数据类型与**非基本数据类型**之间的主要区别是：

- Java 的八种基本类型由语言直接定义。引用类型既可以来自 JDK 或第三方库，也可以由程序员自行声明。
- 引用可用于访问对象的字段和方法；基本类型本身不是对象，但可通过自动装箱转换为相应的包装类型，例如

`int` 对应 `Integer`。

- `int` 等基本类型名称是关键字；类名通常按命名惯例以大写字母开头，但这不是区分基本类型与引用类型的语法规则。
- 基本类型变量始终保存相应类型的值；引用类型变量保存引用，也可以保存 `null`。对 `null` 调用实例方法会抛出 `NullPointerException`。

非基本类型的示例包括[字符串](#)、[数组](#)、[类](#)等。

Java 字符串

字符串用于存储文本。

`String` 变量包含由双引号（`"`）包围的一组字符：

```
String greeting = "Hello";
```

Java 中的 `String` 实际上是一个对象，这意味着它包含可对字符串执行特定操作的方法。

例如，可以使用 `length()` 方法获取字符串的长度：

```
String txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
System.out.println("The length of the txt string is: " + txt.length());
```

要比较两个字符串，可以使用 `equals()` 方法：

```
String txt1 = "Hello";
String txt2 = "Hello";

String txt3 = "Greetings";
String txt4 = "Great things";

System.out.println(txt1.equals(txt2)); // true
System.out.println(txt3.equals(txt4)); // false
```

`equals()` 比较字符串内容；`==` 比较两个引用是否指向同一个对象，不能用来可靠地比较字符串内容。若变量可能为 `null`，可使用常量字符串调用 `equals()`（例如 `"Hello".equals(txt1)`）或使用 `java.util.Objects.equals(a, b)` 避免 `NullPointerException`。

可以在字符串之间使用 `+` 运算符将它们组合起来。这称为**拼接**：

```
String firstName = "John";
String lastName = "Doe";
System.out.println(firstName + " " + lastName);
```

特殊字符

由于字符串必须写在引号内，Java 会误解下面的字符串并产生错误：

```
String txt = "We are the so-called "Vikings" from the north.";
```

避免此问题的方法是使用**反斜杠转义字符**。

反斜杠（\）转义字符会将特殊字符转换为字符串字符：

转义字符	结果	说明
'	'	单引号
"	"	双引号
\	\	反斜杠

Java 中其他常见且有效的转义序列包括：

代码	结果
\n	换行
\t	制表符
\b	退格
\r	回车
\f	换页

Java var 受限类型名

`var` 在 Java 10（2018 年发布）中作为局部变量类型推断使用。严格来说，`var` 是受限类型名，而不是普通关键字。

`var` 关键字允许编译器根据你赋给变量的值自动检测变量类型。

这有助于编写更简洁的代码并避免重复类型，尤其适用于较长或复杂的类型。

例如，可以不写 `int x = 5;`，而写成：

```
var x = 5; // x is an int
System.out.println(x);
```

`var` 只能用于具有初始化器的局部变量，以及部分循环变量等允许的局部声明位置；不能用于字段、方法参数或方法返回类型，也不能写成没有初始化器的 `var x;`。编译器会在编译期确定一个静态类型，`var` 不会让 Java 变成动态类型语言，变量类型之后也不会改变。

Java 数组

数组用于在单个变量中存储多个相同元素类型的值，而不必为每个值声明单独的变量。数组创建后长度固定；如果需要可变长度集合，通常使用 `ArrayList` 等集合类型。

要声明数组，请使用**方括号** `[]` 定义变量类型：

```
String[] cars;
```

现在，我们已经声明了一个保存字符串数组的变量。要向其中插入值，可以将以逗号分隔的值列表放在**花括号** `{ }` 内：

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

访问数组元素

可以通过引用索引号访问数组元素。

此语句访问 cars 中第一个元素的值：

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars[0]);
// Outputs Volvo
```

说明：数组索引从 0 开始：[0] 是第一个元素，[1] 是第二个元素，依此类推。

访问小于 0 或大于等于 `length` 的索引会在运行时抛出 `ArrayIndexOutOfBoundsException`。

更改数组元素

要更改特定元素的值，请引用其索引号：

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
cars[0] = "Opel";
System.out.println(cars[0]);
// Now outputs Opel instead of Volvo
```

`new` 关键字

也可以使用 `new` 指定数组大小来创建数组。这会创建一个空数组，为固定数量的元素预留空间，稍后可以填充这些元素：

```
String[] cars = new String[4]; // size is 4

cars[0] = "Volvo";
cars[1] = "BMW";
cars[2] = "Ford";
cars[3] = "Mazda";

System.out.println(cars[0]); // Outputs Volvo
```

使用 `new` 创建数组时，每个元素会先取得默认值：整数类型为 `0`，浮点类型为 `0.0`，`boolean` 为 `false`，`char` 为 `\u0000`，引用类型为 `null`。访问尚未赋值的 `String` 数组元素时得到的是 `null`。

数组长度

要了解数组包含多少个元素，请使用 `length` 字段：

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars.length);
// Outputs 4
```

Java 集合概览

数组长度固定。需要动态增删元素时，通常使用 Java Collections Framework：

- `List<E>`：有顺序、可重复，常用实现为 `ArrayList<E>`
- `Set<E>`：不保存重复元素，常用实现为 `HashSet<E>`
- `Map<K, V>`：保存键值映射，常用实现为 `HashMap<K, V>`；`Map` 不继承 `Collection`

```
import java.util.ArrayList;
import java.util.List;

List<String> teams = new ArrayList<>();
teams.add("FRC 6907");
teams.add("FRC 254");

System.out.println(teams.get(0));
System.out.println(teams.size());
```

通常应把变量声明为所需的接口类型（如 `List<String>`），再选择具体实现（如 `ArrayList<>`）。集合使用泛型保证元素类型安全；基本类型需要使用包装类型，例如 `List<Integer>`，不能写成 `List<int>`。使用基于哈希的 `HashSet` 或 `HashMap` 存放自定义对象时，必须正确且一致地实现 `equals()` 和 `hashCode()`。

Java 运算符

运算符用于对变量和值执行操作。

Java 算术运算符

算术运算符用于执行常见的数学运算。

运算符	名称	说明	示例
+	加法	将两个值相加	<code>x + y</code>
-	减法	从一个值中减去另一个值	<code>x - y</code>
*	乘法	将两个值相乘	<code>x * y</code>
/	除法	用一个值除以另一个值	<code>x / y</code>
%	取模	返回除法余数	<code>x % y</code>
++	递增	将变量值增加 1	<code>++x</code>
--	递减	将变量值减少 1	<code>--x</code>

说明：在 Java 中两个整数相除时，结果也将是整数。例如，`10 / 3` 得到 `3`。如果需要小数结果，请使用 `double` 值，例如 `10.0 / 3`。

整数除以 0 会抛出 `ArithmeticException`；浮点数除以 0 通常产生 `Infinity`、`-Infinity` 或 `NaN`。整数运算溢出时会按固定位宽回绕而不会自动报错，需要检测溢出时可使用 `Math.addExact()`、`Math.multiplyExact()` 等方法。

```
int a = 10;
int b = 3;
System.out.println(a / b);    // Integer division, result is 3

double c = 10.0d;
double d = 3.0d;
System.out.println(c / d);    // Decimal division, result is 3.333...
```

递增和递减

递增和递减在编程中非常常见，尤其是在使用计数器、循环和数组时（后续章节将进一步介绍）。

`++` 运算符将值增加 1，而 `--` 运算符将值减少 1：

```
int x = 5;

++x; // Increment x by 1
System.out.println(x); // 6
```

```
int x = 5;

--x; // Decrement x by 1
System.out.println(x); // 4
```

赋值运算符

赋值运算符用于给变量赋值。以下是所有赋值运算符：

运算符	示例	等同于
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

说明：大多数赋值运算符只是代码的简写形式。例如，`x += 5` 等同于 `x = x + 5`，但更短且通常更易读。

Java 比较运算符

比较运算符用于比较两个值（或变量）。这在编程中很重要，因为它有助于得出答案并作出决定。

比较的返回值为 `true` 或 `false`。这些值称为布尔值，布尔值和 If..Else 章节将进一步介绍它们。

以下是所有比较运算符：

运算符	名称	示例
<code>==</code>	等于	<code>x == y</code>
<code>!=</code>	不等于	<code>x != y</code>
<code>></code>	大于	<code>x > y</code>
<code><</code>	小于	<code>x < y</code>
<code>>=</code>	大于或等于	<code>x >= y</code>
<code><=</code>	小于或等于	<code>x <= y</code>

Java 逻辑运算符

与比较运算符一样，也可以使用逻辑运算符测试 `true` 或 `false` 值。

逻辑运算符通过组合多个条件来确定变量或值之间的逻辑关系：

运算符	名称	说明	示例
<code>&&</code>	逻辑与	如果两个语句均为 true，则返回 true	<code>x < 5 && x < 10</code>
<code> </code>	逻辑或	如果其中一个语句为 true，则返回 true	<code>x < 5 x < 4</code>
<code>!</code>	逻辑非	反转结果；如果结果为 true，则返回 false	<code>!(x < 5 && x < 10)</code>

Java 运算符优先级

当计算中包含多个运算符时，Java 会遵循**运算顺序**规则来决定先计算哪一部分。

以下是一些常见运算符，按优先级从高到低排列：

- `()` —— 括号
- `*`、`/`、`%` —— 乘法、除法、取模
- `+`、`-` —— 加法、减法
- `>`、`<`、`>=`、`<=` —— 比较
- `==`、`!=` —— 相等性
- `&&` —— 逻辑与
- `||` —— 逻辑或
- `=` —— 赋值

Java Math

Java Math 类提供了许多方法，可以对数字执行数学运算。

可以使用 `Math.max(x,y)` 方法找出 x 和 y 中的最大值：

```
Math.max(5, 10);
```

可以使用 `Math.min(x,y)` 方法找出 x 和 y 中的最小值：

```
Math.min(5, 10);
```

`Math.sqrt(x)` 方法返回 x 的平方根：

```
Math.sqrt(64);
```

`Math.abs(x)` 方法返回 x 的绝对值（结果通常为非负值）：

```
Math.abs(-4.7);
```

对于整数最小值存在一个重要例外：`Math.abs(Integer.MIN_VALUE)` 仍返回 `Integer.MIN_VALUE`，因为相应的正数超出了 `int` 的表示范围；`long` 的最小值同理。

`Math.pow(x, y)` 方法返回 x 的 y 次幂：

```
Math.pow(2, 8); // 256.0
```

说明：`Math.pow(2, 8)` 表示将 2 自乘 8 次：

$2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 = 256$

说明：`Math.pow()` 方法始终返回 `double`，即使结果是整数。例如，`Math.pow(2, 8)` 返回 256.0（而不是 256）。

舍入方法

Java 提供了多种数值舍入方法：

- `Math.round(x)` ——舍入到最接近的整数值；参数为 `float` 时返回 `int`，参数为 `double` 时返回 `long`
- `Math.ceil(x)` ——向上舍入，并以 `double` 返回大于或等于 x 的最小整数值
- `Math.floor(x)` ——向下舍入，并以 `double` 返回小于或等于 x 的最大整数值

```
Math.round(4.6); // 5
Math.ceil(4.1); // 5.0
Math.floor(4.9); // 4.0
```

布尔表达式

布尔表达式返回一个布尔值：`true` 或 `false`。

这有助于在程序中构建逻辑并作出决定。

例如，可以使用比较运算符（如**大于**（>）运算符）判断表达式（或变量）为 true 还是 false：

```
int x = 10;
int y = 9;

System.out.println(x > y); // Outputs true, because 10 is greater than 9
```

将结果存储在 Boolean 变量中

也可以将比较结果存储在 `boolean` 变量中：

```
int x = 10;
int y = 9;

boolean isGreater = x > y;

System.out.println(isGreater); // Outputs true
```

说明：是将比较结果存储在布尔变量中，还是直接使用比较，由你决定。
存储结果可以提高代码的可读性，尤其是在需要复用结果时。

Java If ... Else

Java 条件和 If 语句

可以使用这些条件针对不同的决定执行不同操作。

Java 提供以下条件语句：

- 如果指定条件为 true，使用 `if` 指定要执行的代码块
- 如果同一条件为 false，使用 `else` 指定要执行的代码块
- 如果第一个条件为 false，使用 `else if` 指定要测试的新条件
- 使用 `switch` 指定多个可供选择的代码块

大多数情况下，条件使用如下比较运算符创建：

- 小于：`a < b`
- 小于或等于：`a <= b`
- 大于：`a > b`
- 大于或等于：`a >= b`
- 等于：`a == b`
- 不等于：`a != b`

条件和 if 语句可用于控制程序流程——决定运行哪些代码以及跳过哪些代码。

可以把它想象成现实生活：*如果下雨，就带伞。否则，什么也不做。*

每条 `if` 语句都需要一个结果为 `true` 或 `false` 的条件。

这意味着 `if` 语句与 `boolean` 值协同工作：

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

```
int x = 20;  
int y = 18;  
if (x > y) {  
    System.out.println("x is greater than y");  
}
```

在上面的示例中，我们使用两个变量 `x` 和 `y`，通过 `>` 运算符测试 `x` 是否大于 `y`。由于 `x` 为 20、`y` 为 18，而 20 大于 18，因此屏幕会打印“x is greater than y”。

else 语句

当 `if` 语句中的条件为 `false` 时，`else` 语句可以运行一个代码块。

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

可以把它想象成现实生活：如果下雨，就带伞。否则 (*else*)，不带伞出门：

```
boolean isRaining = false;  
  
if (isRaining) {  
    System.out.println("Bring an umbrella!");  
} else {  
    System.out.println("No rain today, no need for an umbrella!");  
}
```

由于 `isRaining` 为 `false`，因此不满足 `if` 语句中的条件。这意味着会跳过 `if` 代码块，改为运行 `else` 代码块并打印“No rain today, no need for an umbrella!”。

说明

- `else` 没有条件——它在 `if` 条件为 `false` 时运行。
- 不要在 `if (condition)` 后立即添加分号。这样会提前结束语句，并使 `else` 出现非预期行为。

else if 语句

如果第一个条件为 `false`，使用 `else if` 语句指定要测试的新条件。

```

if (condition1) {
    // block of code to be executed if condition1 is true
} else if (condition2) {
    // block of code to be executed if condition1 is false and condition2 is true
} else {
    // block of code to be executed if both conditions are false
}

```

可以把它想象成现实生活：如果下雨，就带伞。否则，如果（Else if）是晴天，就戴太阳镜。否则，就照常出门。

```

int weather = 2; // 1 = raining, 2 = sunny, 3 = cloudy

if (weather == 1) {
    System.out.println("Bring an umbrella.");
} else if (weather == 2) {
    System.out.println("Wear sunglasses.");
} else {
    System.out.println("Just go outside normally.");
}
// Outputs "Wear sunglasses."

```

另一个示例

此示例根据一天中的时间从三条不同消息中进行选择：

```

int time = 16;

if (time < 12) {
    System.out.println("Good morning.");
} else if (time < 18) {
    System.out.println("Good day.");
} else {
    System.out.println("Good evening.");
}
// Outputs "Good day."

```

`time` 的值为 16。第一个条件（`time < 12`）为 `false`，但第二个条件（`time < 18`）为 `true`。因此，`else if` 代码块运行并打印“Good day.”。

如果 `time` 的值为 22，则没有任何条件为 `true`，程序会改为打印“Good evening.”。

if...else 简写形式

还有一种 [if else](#) 简写形式，它称为**三元运算符**，因为它由三个操作数组成。

它可以用一行代码代替多行代码，最常用于代替简单的 if else 语句：

```
variable = (condition) ? expressionTrue : expressionFalse;
```

不必编写：

```
int time = 20;
if (time < 18) {
    System.out.println("Good day.");
} else {
    System.out.println("Good evening.");
}
```

只需编写：

```
int time = 20;
String result = (time < 18) ? "Good day." : "Good evening.";
System.out.println(result);
```

Java Switch

无需编写许多条 `if..else` 语句，可以改用 `switch` 语句。

可以把它想象成在餐厅点餐：如果选择编号 1，会得到披萨。如果选择 2，会得到汉堡。如果选择 3，会得到意大利面。否则，什么也得不到。

`switch` 语句选择多个代码块之一来执行：

```
switch(expression) {
    case x:
        // code block
        break;
    case y:
        // code block
        break;
    default:
        // code block
}
```

其工作方式如下：

- `switch` 表达式只求值一次。
- 将结果与每个 `case` 值进行比较。
- 如果匹配，则运行匹配的代码块。
- 匹配的 `case` 运行后，`break` 语句会停止 `switch`。
- 如果没有匹配项，则运行 `default` 语句。

在这里使用的传统 `switch` 语句中，如果匹配的 `case` 没有执行 `break`、`return` 或抛出异常，程序会继续执行后续 `case`，这种行为称为贯穿（fall-through）。只有在明确需要贯穿时才应省略 `break`。现代 Java 还支持使用 `case ... ->` 的 `switch` 规则，以避免意外贯穿。

下面的示例使用星期编号计算星期名称：

```
int day = 4;
switch (day) {
    case 1:
```

```

        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
}
// Outputs "Thursday" (day 4)

```

break 关键字

Java 执行到 `break` 关键字时，会跳出 switch 代码块。

这会停止执行该代码块内后续的代码和 case 测试。

找到匹配项并完成任务后，就该跳出了，无需继续测试。

default 关键字

`default` 关键字指定没有 case 匹配时要运行的代码：

```

int day = 4;
switch (day) {
    case 6:
        System.out.println("Today is Saturday");
        break;
    case 7:
        System.out.println("Today is Sunday");
        break;
    default:
        System.out.println("Looking forward to the weekend");
}
// Outputs "Looking forward to the weekend"

```

请注意，如果 `default` 语句是 switch 代码块中的最后一条语句，则不需要 break。

Java 循环

只要指定条件为 true，循环就可以执行一个代码块。

循环非常实用，因为它们可以节省时间、减少错误并提高代码可读性。

警告 (FRC): 不要在 `robotPeriodic()`、Subsystem 的 `periodic()` 或 Command 的 `execute()` 中运行长时间阻塞的循环，也不要使用循环等待传感器达到目标。WPILib scheduler 需要这些方法快速返回，才能继续更新其他 Subsystem、Command、NetworkTables 和安全机制。跨多个调度周期的动作应保存状态并在每次调用时执行少量工作，或使用 Command 的结束条件；同样应谨慎使用 `Thread.sleep()`。

Java While 循环

只要指定条件为 **true**，`while` 循环就会重复执行一个代码块：

```
while (condition) {  
    // code block to be executed  
}
```

在下面的示例中，只要变量 (`i`) 小于 5，循环中的代码就会反复运行：

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

Java Do/While 循环

`do/while` 循环是 `while` 循环的一种变体。此循环在检查条件是否为 **true** 之前，会先执行代码块一次。之后，只要条件为 **true**，它就会重复循环。

```
do {  
    // code block to be executed  
}  
while (condition);
```

下面的示例使用 `do/while` 循环。即使条件为 **false**，循环也始终至少执行一次，因为代码块会在测试条件之前执行：

```
int i = 0;  
do {  
    System.out.println(i);  
    i++;  
}  
while (i < 5);
```

Java For 循环

如果准确知道希望代码块循环多少次，请使用 `for` 循环，而不是 `while` 循环：

```
for (statement 1; statement 2; statement 3) {  
    // code block to be executed  
}
```

语句 1 在执行代码块之前执行（一次）。

语句 2 定义执行代码块的条件。

语句 3 在代码块执行后执行（每次均执行）。

下面的示例将打印数字 0 到 4：

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

- **语句 1** 在循环开始前设置变量：`int i = 0`
- **语句 2** 定义循环运行的条件：`i < 5`。如果条件为 `true`，循环会再次运行；如果为 `false`，循环结束。
- **语句 3** 在每次代码块运行后增加一个值：`i++`

嵌套循环

也可以将一个循环放在另一个循环内。这称为**嵌套循环**。

“外层循环”每迭代一次，“内层循环”就会执行一轮：

```
// Outer loop  
for (int i = 1; i <= 2; i++) {  
    System.out.println("Outer: " + i); // Executes 2 times  
  
    // Inner loop  
    for (int j = 1; j <= 3; j++) {  
        System.out.println(" Inner: " + j); // Executes 6 times (2 * 3)  
    }  
}
```

Java For Each 循环

还有一种增强型 `for`（常称“**for-each**”）循环，用于遍历**数组**或实现了 `Iterable` 的对象（例如多数 `Collection`）：

```
for (type variableName : arrayName) {  
    // code block to be executed  
}
```

for-each 循环比常规 `for` 循环更简单、更易读，因为不需要计数器（如 `i < array.length`）。

以下示例打印 `cars` 数组中的所有元素：

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
  
for (String car : cars) {  
    System.out.println(car);  
}
```

说明：当只需要读取元素时，**for-each** 循环非常适用。给循环变量重新赋值不会修改数组中的对应元素；如果要替换元素或跟踪索引，请改用常规 **for** 循环。对于集合，在 **for-each** 循环中直接增删元素还可能触发

`ConcurrentModificationException`，应改用 `Iterator` 或集合提供的安全操作。

Java Break 和 Continue

Break

在本教程前面的章节中，你已经见过 `break` 语句。它用于“跳出” `switch` 语句。

`break` 语句也可用于跳出**循环**。

当 `i` 等于 4 时，此示例会停止循环：

```
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        break;
    }
    System.out.println(i);
}
```

Continue

如果出现指定条件，`continue` 语句会跳过当前迭代中尚未执行的语句，然后继续下一次循环迭代。

此示例会跳过值 4：

```
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        continue;
    }
    System.out.println(i);
}
```

要点：

- `break` = 完全停止循环。
- `continue` = 跳过本轮，但继续循环。

Java 方法

方法是一段仅在被调用时才运行的代码。

可以将称为参数的数据传递给方法。

方法用于执行特定操作，也称为**函数**。

为什么使用方法？为了复用代码：只定义一次代码，然后多次使用。

创建方法

方法必须在类中声明。其定义方式是方法名称后跟括号 `()`。Java 提供了一些预定义方法，例如

`System.out.println()`；你也可以创建自己的方法来执行特定操作：

```
public class Main {  
    static void myMethod() {  
        // code to be executed  
    }  
}
```

示例说明

- `myMethod()` 是方法名称
- `static` 表示该方法属于 `Main` 类，而不属于 `Main` 类的对象。本教程稍后将进一步介绍对象以及如何通过对象访问方法。
- `void` 表示此方法没有返回值。本章稍后将进一步介绍返回值

调用方法

要在 Java 中调用方法，请写出方法名称，后跟一对括号 `()` 和一个分号 `;`。

在以下示例中，调用 `myMethod()` 时，它用于打印一段文本（该操作）：

```
public class Main {  
    static void myMethod() {  
        System.out.println("I just got executed!");  
    }  
  
    public static void main(String[] args) {  
        myMethod();  
    }  
}  
  
// Outputs "I just got executed!"
```

Java 方法参数

可以将信息传递给方法。方法声明中的变量称为**形参 (parameter)**，调用方法时传入的具体值称为**实参 (argument)**。形参在方法内部充当局部变量。

参数在方法名之后的括号内指定。可以添加任意数量的参数，只需用逗号分隔。

以下示例包含一个方法，该方法接受名为 **fname** 的 `String` 参数。调用该方法时，我们传入名字，并在方法内部使用它来打印全名：

```

public class Main {
    static void myMethod(String fname) {
        System.out.println(fname + " Refsnes");
    }

    public static void main(String[] args) {
        myMethod("Liam");
        myMethod("Jenny");
        myMethod("Anja");
    }
}

```

可以包含任意数量的参数：

```

public class Main {
    static void myMethod(String fname, int age) {
        System.out.println(fname + " is " + age);
    }

    public static void main(String[] args) {
        myMethod("Liam", 5);
        myMethod("Jenny", 8);
        myMethod("Anja", 31);
    }
}

```

参数传递是按值传递

Java 始终按值传递（pass-by-value）。对于基本类型，方法收到的是值的副本；对于对象，方法收到的是“对象引用这个值”的副本。方法可以通过该引用修改同一个可变对象，但把形参重新赋给另一个对象不会改变调用方变量保存的引用。Java 不会把变量本身按引用传递。

Java 返回值

在上一页中，所有示例都使用了 `void` 关键字（例如 `static void myMethod(int x)`），它表示方法不返回值。

如果希望方法返回一个值，可以使用基本数据类型（例如 `int`、`char` 等）代替 `void`，并在方法内使用 `return` 关键字：

```

public class Main {
    static int myMethod(int x) {
        return 5 + x;
    }

    public static void main(String[] args) {
        System.out.println(myMethod(3));
    }
}
// Outputs 8 (5 + 3)

```

Java 方法重载

通过**方法重载**，多个方法可以具有相同的名称和不同的参数：

```
int myMethod(int x)
float myMethod(float x)
double myMethod(double x, double y)
```

在下面的示例中，我们重载 `plusMethod` 方法，使其同时适用于 `int` 和 `double`：

```
static int plusMethod(int x, int y) {
    return x + y;
}

static double plusMethod(double x, double y) {
    return x + y;
}

public static void main(String[] args) {
    int myNum1 = plusMethod(8, 5);
    double myNum2 = plusMethod(4.3, 6.26);
    System.out.println("int: " + myNum1);
    System.out.println("double: " + myNum2);
}
```

说明：只要参数的数量和/或类型不同，多个方法就可以使用相同名称。

Java 作用域

在 Java 中，只能在变量创建所在的区域内访问变量。这称为**作用域**。

方法作用域

直接在方法中声明的变量，从其声明所在代码行之后开始，可在方法中的任何位置使用：

```
public class Main {
    public static void main(String[] args) {

        // Code here CANNOT use x

        int x = 100;

        // Code here CAN use x
        System.out.println(x);
    }
}
```

块作用域

代码块是指花括号 `{ }` 之间的所有代码。

在代码块中声明的变量，只能由花括号之间且位于变量声明行之后的代码访问：

```

public class Main {
    public static void main(String[] args) {

        // Code here CANNOT use x

        { // This is a block

            // Code here CANNOT use x

            int x = 100;

            // Code here CAN use x
            System.out.println(x);

        } // The block ends here

        // Code here CANNOT use x

    }
}

```

循环作用域

在 `for` 循环内声明的变量仅存在于该循环内：

```

public class Main {
    public static void main(String[] args) {

        for (int i = 0; i < 5; i++) {
            System.out.println(i); // i is accessible here
        }

        // i is NOT accessible here
    }
}

```

- `for` 循环有自己的代码块（`{ ... }`）。
- 在循环头（`int i = 0`）中声明的变量 `i` 只能在该循环代码块内访问。
- 循环结束后，`i` 超出作用域，因此不能在循环外通过该名称访问。它在运行时何时释放并不是由源码作用域直接规定的。

Java 递归

递归是一种让方法调用自身的技术。该技术提供了一种将复杂问题分解为更易解决的简单问题的方法。

递归可能有些难以理解。弄清其工作原理的最佳方式是亲自试验。

在以下示例中，递归通过将问题分解为两个数字相加这一简单任务，把某个范围内的数字相加：

```

public class Main {
    public static int sum(int k) {
        if (k > 0) {

```

```

        return k + sum(k - 1);
    } else {
        return 0;
    }
}

public static void main(String[] args) {
    int result = sum(10);
    System.out.println(result);
}
}

```

调用 `sum()` 方法时，它将参数 `k` 与所有小于 `k` 的数字之和相加，并返回结果。当 `k` 变为 0 时，该方法只返回 0。程序运行时遵循以下步骤：

```

10 + sum(9)
10 + ( 9 + sum(8) )
10 + ( 9 + ( 8 + sum(7) ) )
...
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0

```

由于 `k` 为 0 时该方法不会再调用自身，程序会在此停止并返回结果。

终止条件

正如循环可能遇到无限循环问题一样，递归方法也可能遇到无限递归问题。无限递归是指方法永不停止调用自身。每个递归方法都应具有终止条件，即方法停止调用自身的条件。在上一个示例中，终止条件是参数 `k` 变为 0。

每次递归调用通常都会占用线程调用栈空间。递归层数过深，即使最终存在终止条件，也可能抛出 `StackOverflowError`。对于可能处理大量元素的简单线性递归，循环通常更安全。

Java 异常处理

异常表示程序执行过程中出现的异常情况。可以使用 `try`、`catch` 和可选的 `finally` 处理异常；使用 `throw` 主动抛出异常，使用 `throws` 在方法声明中说明可能传播的受检异常。

```

try {
    int number = Integer.parseInt("not a number");
    System.out.println(number);
} catch (NumberFormatException exception) {
    System.out.println("Invalid number");
} finally {
    System.out.println("Parsing finished");
}

```

- **受检异常 (checked exception)** 通常表示调用方可以处理的外部失败；编译器要求捕获它或使用 `throws` 声明，例如部分文件 I/O 异常。
- **非受检异常 (unchecked exception)** 是 `RuntimeException` 的子类，通常表示编程错误、无效输入或状态问题，例如 `NullPointerException` 和 `IllegalArgumentException`。
- `Error` 通常表示 JVM 或运行环境的严重问题，例如 `OutOfMemoryError`；一般不应把它当作普通业务异常捕

获后继续运行。

不要使用空的 `catch` 静默忽略异常。应捕获能够实际处理的最具体异常，并保留足够的上下文用于诊断。对于实现 `AutoCloseable` 的文件、流等资源，优先使用 `try-with-resources`，确保资源在成功或失败时都能关闭。

警告（FRC）：机器人主循环或 `scheduler` 线程中的未捕获异常可能中断关键周期逻辑。处理异常时既不要让机构在未知状态下继续输出，也不要只吞掉异常；应记录故障、进入明确的安全状态，并通过仿真或实机测试验证恢复行为。

Java——什么是 OOP？

OOP 是**面向对象编程（Object-Oriented Programming）**的缩写。

过程式编程侧重于编写对数据执行操作的过程或方法，而面向对象编程侧重于创建同时包含数据和方法的对象。

OOP 非常适合构建可扩展、可复用且易于维护的代码。

与过程式编程相比，面向对象编程具有多项优势：

- OOP 为程序提供清晰的结构
- OOP 有助于使 Java 代码遵循 DRY（“Don't Repeat Yourself”，不要重复自己）原则，并使代码更易维护、修改和调试
- OOP 能够以更少的代码和更短的开发时间创建完全可复用的应用程序

提示：“Don't Repeat Yourself”（DRY）原则旨在减少代码重复。应提取应用程序中共用的代码，将其放在一个位置并进行复用，而不是重复编写。

OOP 并不意味着所有复用都应通过继承实现。只有存在稳定的“is-a”关系并需要多态替换时才应优先考虑继承；对于对象协作和可替换依赖，组合通常耦合更低。在 WPILib Command-Based 项目中，Subsystem、Command 和辅助组件之间也应保持清晰职责，避免为了复用少量代码建立过深的继承层次。

Java——什么是类和对象？

类和对象是面向对象编程的两个主要方面。Java 中的对象具有由类声明的字段和方法。例如：在现实生活中，汽车可视为一个对象，它具有重量和颜色等**状态**，也具有行驶和制动等**行为**。

类定义对象可具有的实例字段和方法，对象是类在运行时的实例。继承发生在类与类之间，而不是“对象继承类”；对象可以使用其类及父类中对它可见的成员。

创建类

要创建类，请使用关键字 `class`。

在本示例中，我们创建一个名为 `Main` 的类，其中包含一个字段 `x`：

```
public class Main {  
    int x = 5;  
}
```

创建对象

在 Java 中，对象由类创建。定义类之后，可以使用 `new` 关键字从该类创建对象：

```
public class Main {  
    int x = 5;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        System.out.println(myObj.x);  
    }  
}
```

使用多个类

也可以创建某个类的对象，并在另一个类中访问它。这通常用于更好地组织类（一个类包含相关字段和方法，而另一个类包含 `main()` 方法，即要执行的代码）。

请记住，Java 文件名应与类名一致。在本示例中，我们在同一目录/文件夹中创建了两个文件：

- Main.java
- Second.java

```
public class Main {  
    int x = 5;  
}
```

```
class Second {  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        System.out.println(myObj.x);  
    }  
}
```

编译两个文件：

```
C:\Users\Your Name>javac Main.java  
C:\Users\Your Name>javac Second.java
```

运行 Second.java 文件：

```
C:\Users\Your Name>java Second
```

输出将为 5。

Java 类字段

在上一章中，我们将示例中的 `x`（如下所示）称为“变量”。

在 Java 语言规范和常见技术资料中，在类中声明的变量称为**字段（field）**。有些入门资料也将其宽泛地称为“属性（attribute）”；本文后续统一使用“字段”。

访问字段

可以通过创建类的对象并使用点语法（`.`）来访问可见字段：

以下示例会创建一个名为 `myObj` 的 `Main` 类对象。我们使用该对象的 `x` 字段打印其值：

```
public class Main {
    int x = 5;

    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println(myObj.x);
    }
}
```

修改字段

如果字段不是 `final` 且访问权限允许，也可以修改字段值：

```
public class Main {
    int x;

    public static void main(String[] args) {
        Main myObj = new Main();
        myObj.x = 40;
        System.out.println(myObj.x);
    }
}
```

如果不希望字段引用或基本类型值被再次赋值，请将字段声明为 `final`：

```
public class Main {
    final int x = 10;

    public static void main(String[] args) {
        Main myObj = new Main();
        myObj.x = 25; // will generate an error: cannot assign a value to a final variable
        System.out.println(myObj.x);
    }
}
```

Java 类方法

你已在 Java 方法章节中了解到，方法在类中声明，并用于执行特定操作：

```
// Create a Main class
public class Main {

    // Create a fullThrottle() method
    public void fullThrottle() {
        System.out.println("The car is going as fast as it can!");
    }

    // Create a speed() method and add a parameter
    public void speed(int maxSpeed) {
```

```

        System.out.println("Max speed is: " + maxSpeed);
    }

    // Inside main, call the methods on the myCar object
    public static void main(String[] args) {
        Main myCar = new Main();    // Create a myCar object
        myCar.fullThrottle();        // Call the fullThrottle() method
        myCar.speed(200);            // Call the speed() method
    }
}

// The car is going as fast as it can!
// Max speed is: 200

```

示例说明

1. 我们使用 `class` 关键字创建了自定义 `Main` 类。
2. 我们在 `Main` 类中创建了 `fullThrottle()` 和 `speed()` 方法。
3. 调用 `fullThrottle()` 方法和 `speed()` 方法时，它们会打印一些文本。
4. `speed()` 方法接受一个名为 `maxSpeed` 的 `int` 参数——我们将在第 **8**) 步中使用它。
5. 为了使用 `Main` 类及其方法，需要创建 `Main` 类的一个**对象**。
6. 然后转到 `main()` 方法。你现在已经知道，它是运行程序的 Java 内置方法（main 内的任何代码都会执行）。
7. 我们使用 `new` 关键字创建了一个名为 `myCar` 的对象。
8. 然后，我们在 `myCar` 对象上调用 `fullThrottle()` 和 `speed()` 方法，并使用对象名称（`myCar`），后跟一个点（`.`），再后跟方法名称（`fullThrottle();` 和 `speed(200);`）来运行程序。请注意，我们在 `speed()` 方法中添加了一个值为 **200** 的 `int` 参数。

请记住.....

点（`.`）用于访问对象的可见字段和方法。

要在 Java 中调用方法，请写出方法名称，后跟一对括号（`()`），再后跟分号（`;`）。

`public` 顶层类必须具有与之匹配的文件名（例如 `Main` 和 `Main.java`）。

Java 构造函数

Java 中的构造函数是一种用于初始化对象的**特殊方法**。

创建类的对象时，会调用构造函数。

它可用于设置对象字段的初始值：

```

// Create a Main class
public class Main {
    int x;    // Create a class attribute

    // Create a class constructor for the Main class
    public Main() {
        x = 5;    // Set the initial value for the class attribute x
    }
}

```

```

public static void main(String[] args) {
    Main myObj = new Main(); // Create an object of class Main (This will call the
    constructor)
    System.out.println(myObj.x); // Print the value of x
}
}

// Outputs 5

```

请注意，构造函数名称必须**与类名一致**，并且不能具有**返回类型**（如 `void`）。

另请注意，创建对象时会调用构造函数。

如果一个类没有显式声明任何构造函数，编译器会为它提供一个默认的空参构造函数。该构造函数会先调用父类的空参构造函数；字段仍可通过字段初始化器或实例初始化块设置初始值。一旦显式声明了任意构造函数，编译器就不会再自动提供默认空参构造函数。

构造函数参数

构造函数也可以接受参数，这些参数可用于初始化字段。

以下示例向构造函数添加一个 `int y` 参数。在构造函数内部，我们将 `x` 设为 `y` (`x=y`)。调用构造函数时，我们向构造函数传入参数 `(5)`，这会将 `x` 的值设为 `5`：

```

public class Main {
    int x;

    public Main(int y) {
        x = y;
    }

    public static void main(String[] args) {
        Main myObj = new Main(5);
        System.out.println(myObj.x);
    }
}

// Outputs 5

```

Java this 关键字

Java 中的 `this` 关键字指向方法或构造函数中的当前对象。

当类字段与方法或构造函数参数同名时，经常使用 `this` 关键字来避免混淆。

访问类字段

有时，构造函数或方法的参数与类变量同名。发生这种情况时，该参数会在该方法或构造函数内部暂时**遮蔽**类变量。

要引用类变量而不是参数，可以使用 `this` 关键字：

```

public class Main {
    int x; // Class variable x
}

```

```

// Constructor with one parameter x
public Main(int x) {
    this.x = x; // refers to the class variable x
}

public static void main(String[] args) {
    // Create an object of Main and pass the value 5 to the constructor
    Main myObj = new Main(5);
    System.out.println("Value of x = " + myObj.x);
}
}

```

从另一个构造函数调用构造函数

也可以使用 `this()` 调用同一类中的另一个构造函数。

如果想提供默认值或复用初始化代码而不是重复编写，此功能非常有用。

在 Java 17（FRC 2026 工具链使用的语言级别）中，对 `this()` 或 `super()` 的显式调用必须是构造函数体中的第一条语句，因此二者不能在同一个构造函数中同时显式调用。Java 25 的 flexible constructor bodies 放宽了“必须是第一条语句”的限制，但仍对父类构造完成前可执行的操作施加约束；FRC 项目应以当赛季模板配置的 Java 版本为准。

```

public class Main {
    int modelYear;
    String modelName;

    // Constructor with one parameter
    public Main(String modelName) {
        // Call the two-parameter constructor to reuse code and set a default year
        this(2020, modelName);
    }

    // Constructor with two parameters
    public Main(int modelYear, String modelName) {
        // Use 'this' to assign values to the class variables
        this.modelYear = modelYear;
        this.modelName = modelName;
    }

    // Method to print car information
    public void printInfo() {
        System.out.println(modelYear + " " + modelName);
    }

    public static void main(String[] args) {
        // Create a car with only model name (uses default year)
        Main car1 = new Main("Corvette");

        // Create a car with both model year and name
        Main car2 = new Main(1969, "Mustang");

        car1.printInfo();
    }
}

```

```
        car2.printInfo();
    }
}
```

Java 修饰符

到现在为止，你已经非常熟悉几乎出现在所有示例中的 `public` 关键字：

```
public class Main
```

`public` 关键字是一个**访问修饰符**，用于设置类、字段、方法和构造函数的访问级别。

修饰符分为两组：

- **访问修饰符**——控制访问级别
- **非访问修饰符**——不控制访问级别，但提供其他功能

访问修饰符

对于**顶层类**，可以使用 `public` 或 `package-private`（不写访问修饰符）访问级别；成员类还可以使用 `private` 或 `protected`：

修饰符	说明
<code>public</code>	任何其他类都可以访问该类
<code>package-private</code>	只有同一 package 中的代码可以访问该顶层类。不指定访问修饰符时使用此访问级别。

对于**字段、方法和构造函数**，可以使用以下修饰符之一：

修饰符	说明
<code>public</code>	所有类都可以访问该代码
<code>private</code>	只有声明该代码的类可以访问该代码
<code>package-private</code>	只有同一 package 中的代码可以访问该成员。不指定访问修饰符时使用此访问级别。
<code>protected</code>	同一 package 中的代码可以访问；其他 package 中的子类也可通过继承关系访问，但不能借任意父类实例访问该成员。

在下面的示例中，该类具有一个 `public` 字段和一个 `private` 字段。

可以把它想象成现实生活：

- `public` ——公共公园，所有人都可以进入
- `private` ——你的房门钥匙，只有你可以使用

```

class Person {
    public String name = "John";    // Public - accessible everywhere
    private int age = 30;           // Private - only accessible inside this class
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        System.out.println(p.name);    // Works fine
        System.out.println(p.age);    // Error: age has private access in Person
    }
}

```

这里，`name` 声明为 `public`，因此可以从 `Person` 类外部访问。而 `age` 声明为 `private`，因此只能在 `Person` 类内部使用。

Java 非访问修饰符

非访问修饰符不控制可见性（不像 `public` 或 `private`），而是为类、方法和字段添加**其他功能**。

最常用的非访问修饰符是 `final`、`static` 和 `abstract`。

如果不希望现有字段被再次赋值，请将字段声明为 `final`：

```

public class Main {
    final int x = 10;
    final double PI = 3.14;

    public static void main(String[] args) {
        Main myObj = new Main();
        myObj.x = 50; // will generate an error: cannot assign a value to a final variable
        myObj.PI = 25; // will generate an error: cannot assign a value to a final variable
        System.out.println(myObj.x);
    }
}

```

`static` 方法属于类，而不属于任何特定对象。这意味着可以在不创建类对象的情况下调用它。

```

public class Main {
    // Static method
    static void myStaticMethod() {
        System.out.println("Static methods can be called without creating objects");
    }

    // Main method
    public static void main(String[] args) {
        myStaticMethod();    // Call the static method
        Main.myStaticMethod(); // Or call it using the class name
    }
}

```

说明：`static` 方法属于类本身。它没有隐式的 `this` 引用，因此不能直接访问实例字段或实例方法；如果显式取得某个对象引用，仍可以通过该对象访问其实例成员。

对于本节讨论的顶层类，常用的非访问修饰符包括 `final` 和 `abstract`：

修饰符	说明
<code>final</code>	其他类不能继承该类
<code>abstract</code>	该类不能直接用于创建对象；要使用抽象类的实例行为，通常由具体子类继承并实现所需抽象方法。

对于**字段和方法**，可以使用以下修饰符之一：

修饰符	说明
<code>final</code>	<code>final</code> 字段在完成初始化后不能再次赋值； <code>final</code> 方法不能被子类覆盖。若字段保存对象引用，不能重新指向其他对象，但对象本身仍可能可变。
<code>static</code>	字段和方法属于类，而不属于某个特定对象。这意味着所有对象共享同一个 <code>static</code> 字段，并且可以在不创建对象的情况下调用 <code>static</code> 方法。
<code>abstract</code>	只能在抽象类中使用，并且只能用于方法。该方法没有方法体，例如 <code>abstract void run();</code> 。方法体由（继承它的）子类提供。

Java 封装

封装是将状态与操作这些状态的行为组合在类中，并通过受控 API 隐藏实现细节。它有助于维护类的不变量，但访问修饰符本身不是安全边界，不能替代输入验证、权限控制或敏感数据保护。常见做法包括：

- 将需要隐藏实现细节的字段声明为 `private`
- 提供 `public` **get** 和 **set** 方法，以访问和更新 `private` 变量的值

Get 和 Set

你已从上一章了解到，`private` 变量只能在同一个类内访问（外部类无法访问）。不过，如果提供 `public` **get** 和 **set** 方法，就可以访问这些变量。

`get` 方法返回变量值，`set` 方法设置该值。

两者的语法均以 `get` 或 `set` 开头，后跟首字母大写的变量名：

```

public class Person {
    private String name; // private = restricted access

    // Getter
    public String getName() {
        return name;
    }

    // Setter
    public void setName(String newName) {
        this.name = newName;
    }
}

```

`get` 方法返回变量 `name` 的值。

`set` 方法接受一个参数 (`newName`)，并将其赋给 `name` 变量。`this` 关键字用于指向当前对象。

但是，由于 `name` 变量声明为 `private`，因此我们**不能**从该类外部访问它：

```

public class Main {
    public static void main(String[] args) {
        Person myObj = new Person();
        myObj.name = "John"; // error
        System.out.println(myObj.name); // error
    }
}

```

由于尝试访问 `private` 变量，我们会收到错误：

```

MyClass.java:4: error: name has private access in Person
    myObj.name = "John";
           ^
MyClass.java:5: error: name has private access in Person
    System.out.println(myObj.name);
                      ^
2 errors

```

我们应改用 `getName()` 和 `setName()` 方法来访问和更新变量：

```

public class Main {
    public static void main(String[] args) {
        Person myObj = new Person();
        myObj.setName("John"); // Set the value of the name variable to "John"
        System.out.println(myObj.getName());
    }
}

```

为什么使用封装？

- 更好地控制类字段的读取和更新规则

- 可以只提供读取方法或更新方法，也可以在方法中进行校验、转换或维护不变量
- 灵活：程序员可以更改代码的一部分而不影响其他部分
- 降低外部代码意外破坏对象状态的风险

Java Package

Java 中的 package 用于对相关类进行分组。可以把它看作**文件目录中的文件夹**。我们使用 package 来避免名称冲突，并编写更易维护的代码。Package 分为两类：

- 内置 Package（来自 Java API 的 package）
- 用户定义的 Package（创建自己的 package）

内置 Package

Java SE API 是随 Java 平台提供的一组标准模块、package、类和接口。具体实现及其许可取决于所使用的 JDK 发行版。

该 API 包含输入输出、集合、并发、网络和数据库访问等组件。应查阅与项目 JDK 版本匹配的 API 文档；例如 [Java SE 17 API](#) 与 FRC 2026 工具链所用的 Java 主版本一致，而学习较新 Java 时应切换到相应版本的文档。

该库按模块、package 和类型组织。可以导入单个类型，也可以按需使用通配符导入某个 package 中的类型；`import` 不会导入子 package。

要在源文件中使用其他 package 中类型的简单名称，通常需要使用 `import` 关键字。`java.lang` 中的类型以及当前 package 中的类型无需显式导入；也可以不导入而直接使用完全限定名：

```
import package.name.Class;    // Import a single class
import package.name.*;       // Import the whole package
```

如果找到想使用的类，例如用于获取用户输入的 `Scanner` 类，请编写以下代码：

```
import java.util.Scanner;
```

在上面的示例中，`java.util` 是一个 package，而 `Scanner` 是 `java.util` package 中的一个类。

要使用 `Scanner` 类，请创建该类的对象，并使用 `Scanner` 类文档中列出的任意可用方法。在本示例中，我们将使用 `nextLine()` 方法读取完整的一行：

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("Enter username");

        String userName = myObj.nextLine();
        System.out.println("Username is: " + userName);
    }
}
```

有许多 package 可供选择。在前面的示例中，我们使用了 `java.util` package 中的 `Scanner` 类。该 package 还包含日期和时间工具、随机数生成器及其他实用类。

要让某个 package 中所有可访问类型都可通过简单名称引用，请在导入声明末尾使用星号（*）。这不会包含子 package，也不会在运行时预先加载这些类。以下示例对 `java.util` 使用按需类型导入：

```
import java.util.*;
```

用户定义的 Package

要创建自己的 package，需要了解 Java 使用文件系统目录来存储 package，就像计算机上的文件夹一样：

```
└─ root
  └─ mypack
    └─ MyPackageClass.java
```

要创建 package，请使用 `package` 关键字：

```
package mypack;
class MyPackageClass {
    public static void main(String[] args) {
        System.out.println("This is my package!");
    }
}
```

按上面的目录结构将文件保存为 `mypack\MyPackageClass.java`。在 `root` 目录中使用 `-d .` 编译，让编译器按 package 结构写入类文件：

```
C:\Users\Your Name\root>javac -d . mypack\MyPackageClass.java
```

在上面的示例中编译 package 后，会创建一个名为“mypack”的新文件夹。要运行 `MyPackageClass.java` 文件，请输入：

```
C:\Users\Your Name\root>java mypack.MyPackageClass
```

Java 继承

在 Java 中，可以让一个类继承另一个类的可继承成员。我们将“继承概念”分为两类：

- **subclass**（子类）——从另一个类继承的类
- **superclass**（父类）——被继承的类

要从一个类继承，请使用 `extends` 关键字。

在下面的示例中，`Car` 类（子类）继承 `Vehicle` 类（父类）的可访问字段和方法：

```
class Vehicle {
    protected String brand = "Ford";           // Vehicle attribute
    public void honk() {                         // Vehicle method
        System.out.println("Tuut, tuut!");
    }
}
```

```

    }
}

class Car extends Vehicle {
    private String modelName = "Mustang";    // Car attribute
    public static void main(String[] args) {

        // Create a myCar object
        Car myCar = new Car();

        // Call the honk() method (from the Vehicle class) on the myCar object
        myCar.honk();

        // Display the value of the brand attribute (from the Vehicle class) and the value of
        the modelName from the Car class
        System.out.println(myCar.brand + " " + myCar.modelName);
    }
}

```

你是否注意到 Vehicle 中的 `protected` 修饰符？

我们将 **Vehicle** 中的 **brand** 字段设置为 `protected` [访问修饰符](#)。如果将其设置为 `private`，`Car` 不能直接访问该字段，但仍可通过 `Vehicle` 提供的方法间接使用它。

final 关键字

如果不希望其他类继承某个类，请使用 `final` 关键字：

```

final class Vehicle {
    ...
}

class Car extends Vehicle {
    ...
}

```

输出将类似于：

```

Main.java:9: error: cannot inherit from final Vehicle
class Main extends Vehicle {
                ^
1 error

```

Java 多态

多态意为“多种形态”，当多个类通过继承相互关联时就会出现多态。

正如上一章所述，**继承**允许子类复用和扩展父类行为。**多态**使相同的父类型引用在运行时调用不同子类的覆盖实现，从而以不同方式执行同一种操作。

例如，设想有一个名为 `Animal` 的父类，其中包含名为 `animalSound()` 的方法。`Animal` 的子类可以是 `Pig`、`Cat`、`Dog`、`Bird`，并且它们也各自实现动物叫声（猪哼叫、喵喵叫等）：

```

class Animal {
    public void animalSound() {
        System.out.println("The animal makes a sound");
    }
}

class Pig extends Animal {
    public void animalSound() {
        System.out.println("The pig says: wee wee");
    }
}

class Dog extends Animal {
    public void animalSound() {
        System.out.println("The dog says: bow wow");
    }
}

```

现在可以创建 `Pig` 和 `Dog` 对象，并在两个对象上调用 `animalSound()` 方法：

```

class Animal {
    public void animalSound() {
        System.out.println("The animal makes a sound");
    }
}

class Pig extends Animal {
    public void animalSound() {
        System.out.println("The pig says: wee wee");
    }
}

class Dog extends Animal {
    public void animalSound() {
        System.out.println("The dog says: bow wow");
    }
}

class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Animal(); // Create a Animal object
        Animal myPig = new Pig(); // Create a Pig object
        Animal myDog = new Dog(); // Create a Dog object
        myAnimal.animalSound();
        myPig.animalSound();
        myDog.animalSound();
    }
}

```

Java super 关键字

在 Java 中，`super` 关键字用于引用子类的父类。

`super` 关键字最常见的用途，是消除父类与子类具有同名方法时产生的混淆。

它主要有两种使用方式：

- 访问父类的字段和方法
- 调用父类构造函数

访问父类方法

如果子类中的方法与父类中的方法同名，可以使用 `super` 调用父类版本：

```
class Animal {
    public void animalSound() {
        System.out.println("The animal makes a sound");
    }
}

class Dog extends Animal {
    public void animalSound() {
        super.animalSound(); // Call the parent method
        System.out.println("The dog says: bow wow");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.animalSound();
    }
}

/*
Output:
The animal makes a sound
The dog says: bow wow
*/
```

访问父类字段

如果父类和子类具有同名字段，也可以使用 `super` 访问父类中的字段：

```
class Animal {
    String type = "Animal";
}

class Dog extends Animal {
    String type = "Dog";

    public void printType() {
        System.out.println(super.type); // Access parent attribute
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.printType();
    }
}
```

调用父类构造函数

使用 `super()` 调用父类的构造函数。这对于复用初始化代码尤其有用。

如果构造函数没有显式调用 `this(...)` 或 `super(...)`，编译器通常会隐式插入无参 `super()`。因此，如果父类没有可访问的无参构造函数，子类构造函数必须显式调用父类某个可访问的构造函数。

```
class Animal {
    Animal() {
        System.out.println("Animal is created");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // Call parent constructor
        System.out.println("Dog is created");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
    }
}
```

Java 抽象

数据**抽象**是隐藏某些细节、仅向用户显示必要信息的过程。

可以使用**抽象类**或**接口**（下一章将进一步介绍）实现抽象。

`abstract` 关键字是一种用于类和方法的非访问修饰符：

- **抽象类**：不能直接实例化，但可以包含字段、构造函数、具体方法和抽象方法，并可作为引用类型使用。
- **抽象方法**：只能在抽象类中使用，并且没有方法体。方法体由（继承它的）子类提供。

抽象类可以同时包含抽象方法和常规方法：

```
abstract class Animal {
    public abstract void animalSound();
    public void sleep() {
        System.out.println("Zzz");
    }
}
```

根据上面的示例，无法创建 `Animal` 类的对象：

```
Animal myObj = new Animal(); // will generate an error
```

抽象类不能直接实例化，但可以作为变量类型、参数类型或返回类型使用。要创建实际对象，需要定义具体子类，并实现尚未实现的抽象方法。

```
// Abstract class
abstract class Animal {
    // Abstract method (does not have a body)
    public abstract void animalSound();
    // Regular method
    public void sleep() {
        System.out.println("Zzz");
    }
}

// Subclass (inherit from Animal)
class Pig extends Animal {
    public void animalSound() {
        // The body of animalSound() is provided here
        System.out.println("The pig says: wee wee");
    }
}

class Main {
    public static void main(String[] args) {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound();
        myPig.sleep();
    }
}
```

为什么以及何时使用抽象类和抽象方法？

抽象类适合表达一组相关类的共同状态和部分实现，同时把必须由子类决定的行为声明为抽象方法。它主要用于建立清晰的抽象边界、减少重复和约束子类 API，而不是提供安全防护。

Java 接口

在 Java 中，实现抽象的另一种方式是使用接口。

`interface` 是一种引用类型，用于声明实现类应遵守的行为约定。接口不是类；现代 Java 接口除抽象方法外，还可以包含 `default` 方法、`static` 方法、`private` 辅助方法、常量和嵌套类型。

```
// interface
interface Animal {
    public void animalSound(); // interface method (does not have a body)
    public void run(); // interface method (does not have a body)
}
```

类使用 `implements` 关键字实现接口。对于接口中没有默认实现的抽象方法，具体实现类必须提供方法体，除非该类本身也声明为 `abstract`：

```
// Interface
interface Animal {
    public void animalSound(); // interface method (does not have a body)
    public void sleep(); // interface method (does not have a body)
}

// Pig "implements" the Animal interface
class Pig implements Animal {
    public void animalSound() {
        // The body of animalSound() is provided here
        System.out.println("The pig says: wee wee");
    }
    public void sleep() {
        // The body of sleep() is provided here
        System.out.println("Zzz");
    }
}

class Main {
    public static void main(String[] args) {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound();
        myPig.sleep();
    }
}
```

接口说明：

- 接口不能直接实例化，但接口类型的变量可以引用实现该接口的对象
- 没有 `default`、`static` 或 `private` 修饰且没有方法体的接口实例方法隐式为 `public abstract`
- 具体实现类必须实现所有尚未获得实现的抽象方法；抽象实现类可以暂不实现
- `default` 方法提供可继承的实例实现，`static` 方法属于接口本身；从 Java 9 起，接口还可使用 `private` 方法复用内部实现
- 接口中声明的字段隐式为 `public`、`static` 和 `final`
- 接口不能包含构造函数（因为它不能用于创建对象）

为什么以及何时使用接口？

1. 用稳定的行为约定降低调用方与具体实现之间的耦合，便于替换实现、测试和扩展。
2. Java 不支持类的多重继承（一个类只能直接继承一个父类），但一个类可以**实现**多个接口，从而同时满足多组行为约定。要实现多个接口，请用逗号分隔它们（参见下面的示例）。接口本身也可以继承多个接口。

多个接口

要实现多个接口，请用逗号分隔它们：

```
interface FirstInterface {
    public void myMethod(); // interface method
}
```

```

interface SecondInterface {
    public void myOtherMethod(); // interface method
}

class DemoClass implements FirstInterface, SecondInterface {
    public void myMethod() {
        System.out.println("Some text..");
    }
    public void myOtherMethod() {
        System.out.println("Some other text...");
    }
}

class Main {
    public static void main(String[] args) {
        DemoClass myObj = new DemoClass();
        myObj.myMethod();
        myObj.myOtherMethod();
    }
}

```

Java 枚举

`enum` 是一种特殊的“类”，表示一组**常量**（不可更改的变量，如 `final` 变量）。

Enum 不只是常量列表：它还可以声明字段、方法和构造函数，并且每个 enum 常量都是该 enum 类型的唯一实例之一。Enum 构造函数不能由外部代码直接调用。

要创建 `enum`，请使用 `enum` 关键字（而不是 `class` 或 `interface`），并用逗号分隔常量。请注意，这些常量应使用大写字母：

```

enum Level {
    LOW,
    MEDIUM,
    HIGH
}

```

可以使用点语法访问 `enum` 常量：

```

Level myVar = Level.MEDIUM;

```

Switch 语句中的 Enum

Enum 经常用于 `switch` 语句，以检查相应的值：

```

enum Level {
    LOW,
    MEDIUM,
    HIGH
}

public class Main {

```

```

public static void main(String[] args) {
    Level myVar = Level.MEDIUM;

    switch(myVar) {
        case LOW:
            System.out.println("Low level");
            break;
        case MEDIUM:
            System.out.println("Medium level");
            break;
        case HIGH:
            System.out.println("High level");
            break;
    }
}

```

遍历 Enum

enum 类型具有一个 `values()` 方法，它返回包含所有 enum 常量的数组。当需要遍历 enum 的常量时，此方法非常有用：

```

for (Level myVar : Level.values()) {
    System.out.println(myVar);
}

```

Java 高级内容

Java 泛型

泛型允许编写适用于不同数据类型的类、接口和方法，而无需预先指定确切类型。

这使代码更加灵活、可复用，并具有类型安全性。

为什么使用泛型？

- **代码复用性**：编写一个适用于不同数据类型的类或方法。
- **类型安全**：在编译时而不是运行时发现类型错误。
- **代码更简洁**：获取对象时无需进行类型转换。

可以使用泛型创建适用于不同数据类型的类：

```

class Box<T> {
    T value; // T is a placeholder for any data type

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}

```

```

}

public class Main {
    public static void main(String[] args) {
        // Create a Box to hold a String
        Box<String> stringBox = new Box<>();
        stringBox.set("Hello");
        System.out.println("Value: " + stringBox.get());

        // Create a Box to hold an Integer
        Box<Integer> intBox = new Box<>();
        intBox.set(50);
        System.out.println("Value: " + intBox.get());
    }
}

```

T 是泛型类型参数。它相当于数据类型的占位符。

- 创建 `Box<String>` 时，**T** 变为 `String`。
- 创建 `Box<Integer>` 时，**T** 变为 `Integer`。

这样，同一个类可以与不同数据类型配合复用，而无需重写代码。

还可以使用泛型创建适用于任意数据类型的方法：

```

public class Main {
    // Generic method: works with any type T
    public static <T> void printArray(T[] array) {
        for (T item : array) {
            System.out.println(item);
        }
    }

    public static void main(String[] args) {
        // Array of Strings
        String[] names = {"Jenny", "Liam"};

        // Array of Integers
        Integer[] numbers = {1, 2, 3};

        // Call the generic method with both arrays
        printArray(names);
        printArray(numbers);
    }
}

```

- `<T>` 是泛型类型参数——它表示该方法可以使用任意类型：`String`、`Integer`、`Double` 等。
- `printArray()` 方法接受一个 `T` 类型的数组，并打印其中的每个元素。
- 调用该方法时，Java 会根据传入的参数确定 `T` 应是什么类型。

当希望编写一个适用于多种类型的方法，而不是为每种类型重复编写代码时，这非常有用。

可以使用 `extends` 关键字限制泛型类或方法可接受的类型。

```
class NumberBox<T extends Number> {
    private final T value;

    NumberBox(T value) {
        this.value = value;
    }

    double doubleValue() {
        return value.doubleValue();
    }
}
```

这里的 `T extends Number` 表示 `T` 必须是 `Number` 或其子类。泛型类型参数不能直接使用 `int` 等基本类型，应使用 `Integer` 等包装类型。Java 泛型主要通过类型擦除实现，因此不能在运行时写 `new T()`，也不能可靠地使用 `instanceof T`。

Java 注解

注解是附加到 Java 声明或类型使用位置上的**元数据**。它们以 `@` 符号开头。

注解本身通常不直接执行逻辑，但编译器、构建工具、注解处理器、测试框架或运行时反射代码可以读取注解并据此改变编译结果、生成代码或影响程序行为。是否能在运行时读取注解取决于其 `@Retention` 策略。

Java 包含多种内置注解。以下是一些最常用的注解：

注解	说明
<code>@Override</code>	表示一个方法覆盖父类方法或实现接口方法
<code>@Deprecated</code>	将方法或类标记为过时或不建议使用
<code>@SuppressWarnings</code>	告知编译器抑制指定类别的编译警告；应尽量缩小使用范围

@Override 注解

`@Override` 注解可帮助编译器检查某个方法是否确实覆盖父类方法或实现接口方法。

它不是必需的，但强烈建议使用，因为它有助于发现错误。

在此示例中，我们明确指出正在覆盖一个方法：

```

class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof!");
    }
}

```

如果意外写错方法名称（例如 `makesound()`），编译器会显示如下错误：

```

class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makesound() { // Typo here!
        System.out.println("Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound();
    }
}

```

```

Main.java:8: error: method does not override or implement a method from a supertype
    @Override
    ^
1 error

```

Java Lambda 表达式

Lambda 表达式在 Java 8 中引入。

Lambda 表达式用于为函数式接口的单个抽象方法提供实现。它可以接受参数，并根据目标方法返回值或执行无返回值操作。Lambda 没有自己的方法名，必须出现在赋值、方法实参、返回值或类型转换等能够确定目标函数式接口的上下文中，不能作为孤立语句存在。

最简单的 Lambda 表达式包含单个参数和一个表达式：

```
parameter -> expression
```

要使用多个参数，请将它们放在括号中：

```
(parameter1, parameter2) -> expression
```

表达式形式的 Lambda 不写 `return`；该表达式可根据目标函数式接口用作返回值，某些语句表达式也可适配无返回值接口。要执行多条语句、循环或条件分支，请使用花括号包围的代码块。如果块形式的 Lambda 需要返回值，请在所有可完成路径上使用 `return`：

```
(parameter1, parameter2) -> {  
    // code block  
    return result;  
}
```

Lambda 通常作为参数传递给方法。例如，可以在 `ArrayList` 的 `forEach()` 方法中使用 Lambda：

```
import java.util.ArrayList;  
  
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Integer> numbers = new ArrayList<Integer>();  
        numbers.add(5);  
        numbers.add(9);  
        numbers.add(8);  
        numbers.add(1);  
        numbers.forEach((n) -> { System.out.println(n); });  
    }  
}
```

变量中的 Lambda

Lambda 表达式可以赋给变量。变量的目标类型必须是只有一个抽象方法的接口（**函数式接口**）；`default`、`static` 方法以及与 `Object` 公共方法匹配的方法不计入该抽象方法数量。Lambda 必须与这个抽象方法的参数和返回类型兼容。

Java 包含许多内置函数式接口，例如经常与集合配合使用的 `Consumer`（来自 `java.util.function` package）。

Lambda 可以捕获外层作用域中的局部变量，但这些变量必须是 `final` 或“事实上为 final”（初始化后未再赋值）。实例字段和静态字段不受这一局部变量捕获规则限制。

```
import java.util.ArrayList;  
import java.util.function.Consumer;  
  
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Integer> numbers = new ArrayList<Integer>();  
        numbers.add(5);  
        numbers.add(9);  
        numbers.add(8);  
        numbers.add(1);  
  
        Consumer<Integer> method = (n) -> { System.out.println(n); };  
    }  
}
```

```
    numbers.forEach(method);  
}  
}
```

将 Lambda 作为方法参数

也可以将 Lambda 表达式传递给方法。该方法的参数必须是函数式接口。随后调用该方法就会运行 Lambda 表达式：

```
@FunctionalInterface  
interface StringFunction {  
    String run(String str);  
}  
  
public class Main {  
    public static void main(String[] args) {  
        StringFunction exclaim = (s) -> s + "!";  
        StringFunction ask = (s) -> s + "?";  
        printFormatted("Hello", exclaim);  
        printFormatted("Hello", ask);  
    }  
  
    public static void printFormatted(String str, StringFunction format) {  
        String result = format.run(str);  
        System.out.println(result);  
    }  
}
```

此程序在 Java 中使用函数式接口和 Lambda 表达式，将行为（函数）作为参数传递给另一个方法。

以下是各组件逐步工作的方式：

```
@FunctionalInterface  
interface StringFunction {  
    String run(String str);  
}
```

只有一个抽象方法的接口称为函数式接口。`@FunctionalInterface` 不是必需的，但它会让编译器检查该约束。`StringFunction` 为任何接受 `String` 输入并返回 `String` 输出的函数定义了约定。

```
StringFunction exclaim = (s) -> s + "!";  
StringFunction ask = (s) -> s + "?";
```

无需编写实现 `StringFunction` 的完整类，Lambda 表达式 `((s) -> ...)` 可以即时创建轻量级实现：

- `exclaim` 接受字符串 `s` 并在末尾追加 `!"`。
- `ask` 接受字符串 `s` 并在末尾追加 `?"`。

两者都与 `StringFunction.run(String str)` 中定义的签名匹配。

```
printFormatted("Hello", exclaim);  
printFormatted("Hello", ask);
```

`printFormatted` 方法接受两个参数：目标 `String` 和一个 `StringFunction` 策略。

```
public static void printFormatted(String str, StringFunction format) {  
    String result = format.run(str);  
    System.out.println(result);  
}
```

在 `printFormatted` 内部：

1. `format.run(str)` 调用传入该方法的具体 Lambda 函数。
2. 对于 `exclaim`，调用 `run("Hello")` 会对 `"Hello" + "!"` 求值，产生 `"Hello!"`。
3. 对于 `ask`，调用 `run("Hello")` 会对 `"Hello" + "?"` 求值，产生 `"Hello?"`。
4. `System.out.println(result)` 将返回的字符串打印到控制台。